sed — MCQ

One correct option per question. No negative marking.

1. *sed* stands for:
   A. Stream EDitor
   B. Simple EDitor
   C. Sequential EDitor
   D. Streaming Encoder

2. Consider **`sed -E 's/([^ ]+) ([^ ]+)/(\2, \1)/'`** applied to a line. What is the effect?
   A. Swaps the first two space-separated fields and inserts a comma (e.g., **`Alice Smith`** → **`Smith, Alice`**)
   B. Swaps the last two fields on the line
   C. Reverses all whitespace-separated fields on the line
   D. Fails unless there are three fields

> **Solution:** The ERE **`([^ ]+)`** captures two non-space tokens; the replacement **`(\2, \1)`** flips them to "2, 1".

3. Which command prints from the first line matching **`BEGIN`** to the *next* line matching **`END`**, inclusive?
   A. **`sed -n '/BEGIN/,/END/p' file`**
   B. **`sed '/BEGIN/,/END/d' file`**
   C. **`sed '/BEGIN/+1,/END/-1p' file`**
   D. **`sed -n '/^BEGIN$/,/^$END/p' file`**

> **Solution:** **`/A/,/B/`** is a range address. With **`-n`**, **`p`** prints only that range. The **`d`** version would delete the range instead.

4. Choose the compact command that, on lines containing "ERROR", first replaces "log" with "LOG" and then deletes those lines:
   A. **`sed '/ERROR/{s/log/LOG/;d}' app.log`**
   B. **`sed -e '/ERROR/s/log/LOG/' -e '/ERROR/p' app.log`**
   C. **`sed 's/log/LOG/;/ERROR/d' app.log`**
   D. **`sed '/ERROR/d; s/log/LOG/' app.log`**

Solution: The block **`/ERROR/{...}`** limits edits to matching lines. After substitution, **`d`** deletes those lines. Other choices either print or modify unintended lines or delete before substituting.

5. Which small script reverses every pair of adjacent lines?

   A. **`sed -n 'h; n; p; g; p' file`**

   B. **`sed 'N; s/\n//; P; D' file`**

   C. **`sed 'x; g; h' file`**

   D. **`sed -n '1!G; h; $p' file`**

Solution: Cycle: copy first line to hold (**h**); read next line (**n**); print it (**p**); restore first from hold (**g**); print it (**p**). Repeats per pair. The last option reverses the *entire* file.

6. What does **`sed '2q' nums.txt`** do?

   A. Prints the first two lines, then quits early (exit status 0)

   B. Prints two lines and sets exit status 2

   C. Prints nothing but sets exit status 2

   D. Prints the whole file twice, then quits

Solution: Address **2** applies **q** on line 2; **q** stops processing immediately after printing the current pattern space by default.

7. You want to write only lines where a substitution happened to a file **`out.txt`**. Which is correct?

   A. **`sed 's/foo/bar/w out.txt' file`**

   B. **`sed 'w out.txt; s/foo/bar/' file`**

   C. **`sed -n 'w out.txt; s/foo/bar/p' file`**

   D. **`sed -n 's/foo/bar/; w out.txt' file`**

Solution: The **w** flag on **`s///`** writes only *when* the substitution succeeds. Standalone **w** writes each (selected) line regardless of substitution.

8. Which option treats each input file as a separate stream (resetting line numbers at each file)?

A. **`-s`**

B. **`-u`**

C. **`-z`**

D. **`y/SET1/SET2/`**

Solution: GNU **`sed -s`** processes files separately; line numbers and addresses like **`1`** or **`$`** apply per file.

9. Which command exchanges the contents of the pattern space and the hold space?

A. **`x`**

B. **`h`**

C. **`H`**

D. **`g`**

Solution: **`x`** swaps; **`h`**/**`H`** copy to hold (replace/append); **`g`**/**`G`** copy from hold (replace/append).

10. What does **`sed -n '=;p' fruits.txt`** do?

A. Prints each line number *and* the corresponding line

B. Prints only line numbers

C. Prints only lines (no numbers)

D. Prints line numbers only for non-empty lines

Solution: **`=`** prints the current input line number; **`p`** then prints the line. With **`-n`**, only these explicit prints appear.

11. In multi-line processing, what does the command **`N`** do?

A. Appends the next input line to the pattern space (separated by "**`\n`**")

B. Replaces the pattern space with the next line

C. Prints the pattern space up to the first newline

D. Deletes text up to the first newline

Solution: **`N`** reads the next input line and appends it to the pattern space with a newline, enabling multi-line regex and substitutions.

12. Which addressing form selects every third line starting from the first line (GNU sed)?

    A. **`1~3`**
    B. **`3n`**
    C. **`%3`**
    D. **`3~1`**

> **Solution:** GNU extension **`addr~step`**; **`1~3`** = 1,4,7,... The other forms are not valid addressing.

13. Which command prints only the lines where "error" was replaced by "ERROR"?

    A. **`sed -n 's/error/ERROR/p' file`**
    B. **`sed 's/error/ERROR/p' file`**
    C. **`sed -n 's/error/ERROR/g' file`**
    D. **`sed 's/error/ERROR/gp' file`**

> **Solution:** The **`p`** flag prints only when the substitution succeeds. With **`-n`**, other lines are suppressed. Without **`-n`**, everything prints (and matched lines print twice).

14. To perform a "global" substitution, replacing every occurrence of a pattern on a line, which flag is used with the substitute command?

    A. The **`a`** flag
    B. The **`r`** flag
    C. The **`g`** flag
    D. The **`*`** flag

> **Solution:** The **`g`** flag, when appended to a substitute command (e.g., **`s/old/new/g`**), stands for "global". It instructs **`sed`** to replace all occurrences of the pattern on the line, rather than just the first one.

15. Which option allows **`sed`** to edit a file "in-place", modifying the original file?

    A. **`-i`**
    B. **`-f`**
    C. **`-o`**
    D. **`-w`**

Solution: The **`-i`** option enables "in-place" editing. When this option is used, **`sed`** modifies the input file directly. You can also provide a suffix (e.g., **`-i.bak`**) to create a backup of the original file before modification.

16. In **`sed`** addressing, which special character is used to refer to the **last line** of the input?

    A. **`^`**

    B. **`$`**

    C. **`&`**

    D. **`!`**

Solution: The dollar sign (**`$`**) is a special address that matches the last line of the input file or stream.

17. Which **`sed`** command is used to **insert** text *before* the current line in the pattern space?

    A. **`a \text`**

    B. **`i \text`**

    C. **`c \text`**

    D. **`r file`**

Solution: The **`i \text`** command **inserts** the specified text before the current line. In contrast, the **`a \text`** command appends text *after* the current line.

18. What is the correct syntax for creating a capturing group in a Basic Regular Expression (BRE), which is the default for **`sed`**?

    A. **`(...)`**

    B. **`\(...\)`**

    C. **`[...]`**

    D. **`<...`**

Solution: In Basic Regular Expressions (BRE), special characters like parentheses for grouping must be escaped with a backslash. The correct syntax is **`\(...\)`**. In Extended Regular Expressions (ERE), activated with **`-E`** or **`-r`**, they are not escaped: **`(...)`**.

19. After an **`N`** command joins two lines (e.g., "A**`\n`**B") in the pattern space, what is the effect of the uppercase **`D`** command?

A. It deletes the entire pattern space ("A**\n**B") and starts a new cycle.

B. It is invalid and will cause an error.

C. It deletes the content up to the first newline ("A**\n**"), leaving "B" in the pattern space, and restarts the cycle.

D. It deletes the second line ("B") that was just appended.

Solution: The **D** command is for multi-line pattern spaces. It deletes the portion of the pattern space up to and including the first newline character and then restarts the cycle *without* reading a new line from input. The **d** command, by contrast, would delete the entire buffer and read a new line.

20. What does the command **sed '2,10!s/foo/bar/' file** do? The **!** character is key here.

A. It applies the substitution **s/foo/bar/** to all lines *except* for lines 2 through 10.

B. It is a syntax error; **!** cannot be used with a line range.

C. It forces the substitution to occur, even if **foo** is not found on lines 2 through 10.

D. It deletes lines 2 through 10 if the substitution fails.

Solution: The exclamation mark (**!**) placed after an address or an address range acts as a negation operator. It inverts the selection, causing the command to be applied to all lines that do *not* match the address.

21. Which **sed** command would you use to find lines containing "ERROR", and then join each of those lines with the line that immediately follows it?

A. **sed '/ERROR/a'**

B. **sed '/ERROR/n'**

C. **sed '/ERROR/{N; s/\n/ /}'**

D. **sed '/ERROR/r next_line.txt'**

Solution: The correct approach is to first find the line with **/ERROR/**, then use the **N** command to append the next line into the pattern space. This creates a multi-line pattern space (e.g., "ERROR line**\n**next line"). A substitute command like **s/\n/ /** can then replace the embedded newline to join the two lines.

22. What does the following script do? **sed -n '1!G; h; $p' file**

A. Deletes every line except the last

B. Reverses the order of all lines

C. Duplicates the first line at the end

D. Prints every line twice

Solution: This is the classic reverse-file idiom in **sed**. It builds the output backwards in the hold space and only prints at the end.
Cycle 1 (Line: "First")

1!G: The command is skipped because this is the first line.

Pattern Space: First

h: The pattern space is copied to the hold space.

Hold Space: First

$p: The command is skipped because it's not the last line.

End of cycle. Nothing is printed.

23. Which command prints every line that is *not the first or last* of the file?

A. `sed '1d; $d' file`

B. `sed -n '2,${p}' file`

C. `sed -n '1!{$!p}' file`

D. All of the above

Solution: All three patterns achieve the same: suppress printing of the first and last line.

24. Which script converts lowercase to uppercase only in lines containing "error", leaving other lines untouched?

A. `sed '/error/ y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVW`

B. `sed -n '/error/ y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRST`

C. `sed 'y/abcdefghijklmnopqrstuvwxyz/ABCDEFGHIJKLMNOPQRSTUVWXYZ/'`

D. `sed '/error/s/[a-z]/\U&/g'`

Solution: Using address `/error/` limits the transliteration `y///` to only those lines.

25. How many lines will be printed by:

```
seq 1 5 | sed 'N; D'
```

A. 1

B. 2

C. 3

D. 5

Solution: It outputs roughly half the lines; for 5 input lines, 3 lines are printed (lines 2,4,5).